图麦笔记

> 让有价值的信息，不止于收藏

比赛提交说明

> 一款把链接、截图和手写内容变成知识笔记的微信小程序。一条链接、一张截图、一段手写文字丢进来，十几秒后回来的是一条带标题、摘要、核心要点和标签的笔记——之后能搜、能归类、能生成一张带小程序码的卡片图随时调用。

> 下面依次回答：这个需求是怎么被我撞上的；我打算用什么不一样的方式解它；技术上怎么把它搭起来（这一层是重点）；界面为什么长成这样；以及一个个人开发者为什么能在六天内把它推到线上。

01 · 需求发现与应用场景

需求不是想出来的，是我自己撞上的

我做产品经理这些年，有一个习惯改不掉：看到有价值的东西先存下来。

存的地方有三个——微信收藏里躺着两百多条公众号文章，相册里有一个专门的"截图"相簿，四百多张；还有一类更零散的，是群里别人丢过来的报告链接和聊天里截的半张图。

真正需要用到它们的时候，我做过一次自我统计：为了找一份"上周看过的关于获客成本的行业文章"，我翻了收藏、翻了相册、又翻了三个群，最后没找到，索性重新搜了一遍关键词。**那篇文章我确实"保存"过，但我保存的只是一个入口，不是内容本身。**

这件事让我意识到三个事实：

第一，收藏动作的完成率极高，复用率接近于零。 点"收藏"和按截图键只需要一秒钟，而这一下点击背后隐含的承诺——"我以后会整理、会用到"——几乎从来没有被兑现。

第二，笔记工具并没有解决这个问题，它甚至加剧了问题。 所有笔记应用的第一屏都是一块空白编辑器，它在无声地要求用户先回答"你要写什么"。可是碎片信息场景里，用户手里有的不是想法，是**别人的内容**。从"我有一段别人的内容"到"我有一条自己的笔记"，中间那次摘录、概括、打标签的活，至今完全由人肉完成——而这一步恰恰是最累的，所以它总是被跳过。

第三，这些内容本来就发生在微信里。 一篇公众号文章、一张聊天截图、一个群文件链接。任何要求我"复制到另一个 App 去整理"的方案，在启动成本上都先天输一场。

于是需求被我压缩成一句话：**把"存下来"和"整理好"之间那一步，交给机器。**

而这件事在两年前还不成立，现在成立，是因为一个前提变了——**大模型第一次便宜到个人开发者可以随手用。**这个前提有多便宜，我在第三节用现网读数回答：把一篇长文提炼成结构化笔记，实测每篇的现金成本接近 **0 元**。

02 · 产品设计与创新选择

规划产品时，我刻意做了几个不一样的选择

想清楚要做什么之后，真正决定产品形态的是"不做什么"和"把哪里挪一挪"。这些是我在方案阶段就定下来的创新点，后面的架构和界面都是它们的推论。

① 把记录的起点前移：从"空白页"挪到"已经看到的内容"。

这是整个产品最核心的一个位移。用户不需要先想好怎么写，他只需要把手边那条链接、那张截图、那段手写笔记丢进来。启动成本从"组织一段语言"降到"粘贴一次"或"拍一张"，而启动成本正是记录类产品最大的流失点。

② 三种输入，一种结构：链接、截图和手写最后进的是同一个笔记格式。

市面上多数工具是"链接收藏器"或"截图存储空间"，来源不同，存放的地方就不同，找的时候也要分别去翻。我们让抓取、识别和手写这三条完全不同的链路，最后都落到同一组字段上（标题 / 摘要 / 要点 / 标签）。**来源差异被产品刻意抹平，用户的注意力只需要花在"这份内容对我有什么用"。** 其中链接和截图走 AI 提炼，手写内容则保留原始文字由用户自行编辑整理——因为手写本身就是思考过程，不需要机器代劳。



图 1 新建页：URL 导入、截图导入、手动撰写，三条链路最终汇入同一组字段。

③ 出口不只是"在应用里看"。

笔记最有价值的时刻往往不在屏幕里。所以除了搜索、分类、置顶，我们做了一条很少见的路：**把一条笔记生成一张按内容自动排高的卡片图**，插进任意一篇文章当配图，日后从卡片图上的小程序码就能扫回这一条笔记；也可以离线打印出来，贴在纸质手账、攻略本、笔记本上。数字记录的终点，是它被翻到那次是不是方便。

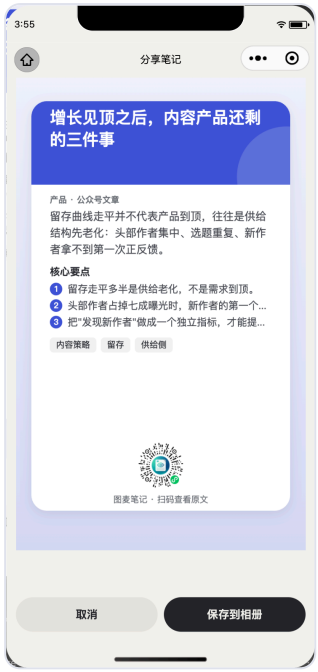


图 2 一条真实笔记导出的卡片图，底部小程序码扫开就是这一条。

④ 克制：界面不许诺服务端还没做的事。

隐私条款里每一句承诺（截图只在内存处理、不落盘、无埋点）都是逐条对着代码核对后才写上去的；HEIC 不支持、经微信转发的图会丢拍摄参数、部分海外站点抓取不稳定，这些边界也如实写在界面里。**"看起来成功了但内容是空的"是最糟糕的失败形态**，所以链接抓不到正文时我们直接报错，不会塞半条空笔记进列表。

⑤ 分享回流设计：七天有效，只露摘要。

笔记可以通过微信分享给朋友，但分享出去的不是笔记全文——对方看到的只是标题、摘要、标签和关键链接这四个公开字段，完整内容仍然留在自己的私密空间里。分享链接有效期七天，过期后卡片图上那个小程序码仍然是唯一入口。**分享是引流，不是泄露。**

这几个选择定了之后，剩下的问题只有一个：**这套东西要拿什么搭出来？**

技术架构：一层免费的 AI 底座，一层开源的确定性能力

3.1 整体分层

端 → 自建后端 → 三条链路。语义线走微信云开发的免费 AI 资源，识别线走自有服务器上的开源模型，抓取线走 trafilatura 直连。

三条链路各自解决一件事：**把内容取回来、把图片变成文字、把文字炼成结构**。设计上有一条硬规矩——**语义线只能被异步任务调用，不被任何同步接口调用**，因为它最慢也最不确定；它挂了就降级存原文，绝不让一次采集整体失败。

3.2 重点：微信免费提供的 AI 能力，把最大的那块成本消掉了

一个个人开发者做"AI 应用"，最先撞上的墙是模型：调第三方大模型要信用卡、要按 token 付钱、要担心被刷爆；自建推理更是想都不用想。这个项目之所以能成立，是因为微信把这道门拆掉了。

（1）模型入口是内建的，不需要任何外部凭据。

微信云开发提供 `cloud.ai()` 这一层统一调用入口，在云函数内部直接可用：**不需要申请第三方 API Key、不需要自建公网出口、不需要维护推理服务，也不需要处理模型侧的鉴权与限流细节**。对开发者来说，"接一个大模型"退化成了在云函数里写一次 `generateText`。

（2）额度是真实免费的，而且我把它跑成了实测结论。

微信"成长计划"给到 **10 亿 Token 的免费额度**。真正有价值的不是这个数字，而是它能不能被用上——这里有一个只有实测才能拿到的结论：

– 控制台里带"免费额度"标记的 `hy3`，一律在 0.68 秒内抛 `AI_MODEL_NOT_ENABLED`，要求先把环境切换为"资源点"套餐，而**那个切换在页面上写明不可回退**；

– 换成 `hy3-preview` 之后，**不切换任何计费形态，调用正常返回，并且免费额度被真实抵扣**。

现网四十余次真实调用的账单读数：**已用 76,248 / 10 亿 Token，而云开发套餐侧的调用次数与计算资源读数同时为 0**。单次提炼折算约 1,900 Token——也就是说，**10 亿额度够五十万次量级的提炼，而有效期之内我们用不完**。把一句话摊开写：**"读懂一篇长文并整理成结构化笔记"这件事，在这个产品里的边际成本接近 0 元。**

（3）长调用的通道是自己趟出来的。

模型一次 1.27–2.87 秒，但整条采集链路（抓取 + 识别 + 提炼）会超过云函数默认 3 秒的超时。做法是把云函数超时抬到 60 秒，并通过 HTTP 触发网关 + 服务端长期 API Key 建立一条"长调用通道"，让自建后端能安全地把一次提炼丢给云函数，同时保证**凭据只出现在服务端配置文件里，不进代码仓库、不进客户端**。

（4）AI 的输出被关进了一份契约。

模型会超时、会被限流、会吐出不合法的内容，所以接口层被设计成"文本进 → 四个字段严格 JSON 出"：`title` / `summary`（2–3 句）/ `key_points`（3–8 条，每条一句话）/ `tags`（3–6 个短词），`temperature 0.3`，输入截断 12,000 字。三个细节值得单说：

– **提示词全仓库只留一份**，由调用方持有并传入云函数。留第二份在云函数里，改一处忘一处，线上就会出现"后端以为换了 prompt、模型其实没换"。

– **可重试性由云函数显式带出来**。实测空闲之后的第一次调用必吃一个 429、紧接着的几次全好；后端最初把所有错误一律当不可重试直接降级，结果每条空闲后的第一条笔记会静默丢掉摘要。现在 429/5xx 走指数退避重试，其余 4xx 直接降级，不浪费时间。

– **降级态是一个可区分的状态，不是一个失败**。提炼不可用时保存原文并标记 `degraded`——"采集失败"和"只是没提炼"是两件事，界面上也绝不会写"已为你生成摘要"。

（5）顺带被免费的还有另一件事。

笔记卡片图上那个可扫的小程序码，用的是微信原生能力（`getwxacodeunlimit`），access_token 在服务端缓存复用；图片识别出的内容要回流给用户，也不需要做 App 分发。**从模型到分发，微信把个人开发者最缺的两样东西——算力和流量入口——都提供了免费版本。**

3.3 开源补齐的另一半：模型做不了的那件事

大模型擅长理解，但它不负责"把图片里的字认出来"。而截图恰恰是这个产品最重要的输入之一。这里必须自建，原因有三层：个人主体拿不到微信官方 OCR 接口；大模型 token 做不了 OCR；以及最关键的——**用户交给我们的是相册里的图片，它不应该再流经任何第三方服务**。

选型落在 **RapidOCR（PP-OCR 系：文本检测 + 方向分类 + 序列识别三模型）+ ONNXRuntime 纯 CPU 推理**：模型随包分发、**零密钥、零按次费用**，加载后常驻内存约 160MB，一台 3.7GB 的单机就跑得起来。图片在后端进程中完成识别后即释放，不额外落盘到对象存储。

但"能跑"和"能在生产上跑不死"之间，隔着三道用实测换来的闸门：

闸门	实测依据
识别前缩放到长边 ≤1600px	同一张 3420×2214 长图：不缩放进程峰值内存 **1,076.6MB**，缩放后 **666.6MB**，而识别出的字符数从 294 **升到** 301——缩放没有丢信息
缩放之前先按像素数拒图（上限 4,000 万像素）	缩放要先解码，所以"单张不超过 10MB"挡不住一张 **0.43MB 的 1.44 亿像素**纯色 PNG（实测吃掉 1,271MB）。现在拒图只花 0.015 秒
按检测框坐标重排为视觉阅读顺序 + 剔除噪声行	开源 OCR 默认按检测顺序输出，长截图会乱序；工具栏图标被认成"AB X""κ7"这类不含中文的极短行，一律丢弃

生产机上的端到端读数：一张 1170×3200 的真实中文长截图，**从提交到笔记出现在列表 4.0 秒**，worker 进程组按 0.15 秒轮询采到的 cgroup 内存峰值 **465.6MB**（当时机器可用 1,959MB）。图片经异步任务队列（Redis + RQ）调度处理，多图批次只在进程内存里暂存，30 分钟未提交自动作废。

至于链接那条路，用的是 **trafilatura + BeautifulSoup/lxml** 在自有服务器上直接抓正文，不经过任何第三方解析服务或短链服务。

3.4 端侧也有自己的技术活

小程序不是后端的皮。有两件事只能在端上做，也值得写：

– **分享卡片图的高度由内容决定。** 画布高度依赖内容、而内容装不装得下又依赖高度，这是个环。做法是把"量"和"画"分成两趟：先测出各块行数算出卡片高度，再落笔。同一张短笔记，卡片从固定 1200 高降到 638，标签和小程序码之间那段五百多行的空白消失了。

– **设计令牌化，一套 UI 适配六套壁纸。** 颜色、间距、圆角、字阶全部收敛成三层令牌（原始值 → 语义值 → 组件值），唯一的色源是一份调色板模块；分类色按语义钉死，同一个功能在全应用只有一张脸。深色壁纸不需要重画一套界面，只需要换 chrome 层的那批变量。

3.5 研发工具链：Qoder + Qwen + 小程序原生框架 + AI

这套交付能压到六天，还有一层原因：**AI 不只是产品里的一个能力，它同时是产品的生产工具。**

整条研发链路建立在一个智能体编程环境（Qoder）之上，推理底座使用 **Qwen 系列模型**，交付运行时是**微信小程序原生框架**，端云之间是自建后端与微信云开发。四者构成的不是"一个会写代码的插件"，而是一条可以自我验证的流水线：

层	承担的事	为什么它让一个人能干完
上下文层	仓库级语义检索与知识卡片：模块边界、约定、踩坑记录被沉淀成可检索的结构	换会话、换人都不丢上下文，改代码前能先读到"这块为什么这么写"
生成层	Qwen 长上下文能力支撑跨文件重构、中文文案与注释的一致性	一次改动能同时覆盖前端页面、样式令牌、后端服务与测试，不留半套
执行层	命令行驱动开发者工具：编译、拉起模拟器、注入自动化端口	改完不必等人去点，机器自己把界面跑起来
验证层	用 `miniprogram-automator` 真点按钮、真输文字、真截图、**跨页面量控件尺寸**；后端 15 个测试文件、230 个测试函数	交付前自己证明"没做坏"，而不是把"你点一下看看"甩回给人

这条流水线里我最满意的一个做法是：**把"一致性"变成可测量的断言**。首页搜索框和新建页输入框必须一样大，那就不是"看着差不多"，而是跨页面各测一次、要求两个数值相等（实测同为 22.4px）；海报"空白没了"这件事，量的必须是导出的成品图逐行扫描，而不是回读画布尺寸——画布尺寸和页面里那个数出自同一份计算，拿它验自己永远是对的。

当前规模：小程序 5,584 行、后端 2,545 行（不含测试）、云函数 123 行；整包 **193.7 KB**；后端 **230 个测试函数**、界面 **55 条自动化断言**。

架构讲完了，但一个能跑的东西不等于一个有人用的东西。下面这部分，是我这些年做产品最贵的那点经验。

04 · 交互设计与界面取舍

交互设计：看过这些国际做法之后，我选了卡片

工具类产品的第一屏决定留存。而"新建笔记"类应用的第一屏，通常都在把认知负担丢回给用户。我在设计这一屏之前，先把国际上几种成熟范式拆开看过，每一种都想清楚"借它什么、放弃什么"：

Bento Grid（苹果式便当盒信息分区）——它的价值是"一格一义"，一个色块只承担一个信息。我借这条，放弃了它那种为展示而生的强装饰网格：手机屏幕上没有纯粹的装饰。

Material 3 的动态取色——它证明颜色可以承载语义而不是只做美化。我借这条，放弃了"颜色跟随主题乱变"：壁纸是用户的偏好，分类色是内容的身份，两者必须分开——**同一分类在这台手机上永远同一个颜色，换壁纸也不变**。

新粗野主义的描边语言——用可见描边取代投影来表达边界。我借了这条，一个很实际的理由：深色背景下投影几乎不可见，而描边在明暗两套皮肤里都成立，一套 UI 不用画两遍。

创作类工具（VSCO / Lightroom / CapCut）的视觉性格——高饱和色块、大圆角、圆润粗字、内容优先。这是我最终定调的方向。我一度按行业惯例做过一版"墨与纸"的克制方案，被直接否掉：在这类产品里，**大胆比冷淡更接近"好用"的感觉**。

Notion / Cubox / Flomo 这一族收藏工具——它们把"存"做得很好，但共同的空白是：**输入端和回流端分属两个生态**。收藏有输入无结构，笔记有结构却拿不到微信相册里的截图。这正是我们挤进去的那道缝。

看了一圈之后，落地的方案我称之为**"左块右文"**，它经历了两版重做才定下来：

– **颜色从整张卡退到卡片左侧那块方块**，卡片本身回到白底描边。第一版是整条笔记就是一张饱和色卡，好看，但一屏三条以后满屏都是色块，读起来累；退一步之后，**扫颜色分流、读文字辨条**，两件事各归其位。

– 方块上写的字，用的是**用户自己打的第一个标签**，而不是分类名——标签更能说明这一条是什么，而颜色已经足够说明它属于哪一类。

– **一个功能只留一个入口**，一屏只允许有一个主行动；导航 tab 用文字加下划线而不是按钮态；卡片上的说明文字一律砍掉，空间优先给内容本身。搜索条是最典型的演化：从"白底胶囊 + 图标按钮"一路减到"一块饱和色 + 一条横线 + 线上几个字"，整块高度从 250rpx 压到 116rpx，**一屏从露出两条半笔记变成三条**。

– 深色模式不是"另一套配色"，而是同一套令牌换一组语义值，六套壁纸里两套深色，所有组件自动成立。

这些取舍背后是同一句话：**好的工具不应该让用户为了管理信息而增加新的负担，它应该让整理更容易发生。**



图 3 当前体验版 v1.1.15 的真实界面：左为笔记列表（左块右文，一屏三条），右为单条笔记详情（摘要、来源链接、原内容折叠、分享为图片）。

05 · 团队情况与验证方法

写在最后：一个人，六天，一台 3.7GB 的服务器

把这件事按传统方式拆一遍，它至少需要一支五人小组：后端工程师、算法或 OCR 方向的人、小程序工程师、视觉设计师、测试。而它现在的情况是——**一个人，六天，一台 3.7GB 内存的服务器，AI 模型零账单，一个 193.7 KB 的线上小程序。**

关键节点只有五个：

时间	节点
09–17	第一版可跑通的闭环：链接、截图、分类、搜索、身份隔离
09–19	OCR 定案自建（个人主体拿不到官方接口，模型 token 做不了识别），同时定下图片在后端进程内完成识别、不额外流经第三方服务这条隐私底线
09–20	后端整批上线现网，混元链路打通，**免费额度的归属被实测证实**；同日完成真机全功能走查
09–21	界面两版重做后定稿"左块右文"，设计令牌化，六套壁纸全量适配
09–22	关于页与十节隐私条款（逐条对代码）、分享回流（七天有效、只露摘要）、页头统计看板；当前体验版 v1.1.15

我想说的不是"AI 帮我写了代码"。

真正变化的是**能力获取的门槛**：模型是免费提供的，识别是开源可用的，开发环境会自己检索上下文并跑验证，交付与分发都在一个国民级入口里完成——过去这些门槛需要一支团队、一份预算和几个月的排期，它们各自挡住的是同一个问题的不同侧面。

而在这条链路上，人的工作没有消失，它上移了一格：**决定契约长什么样、决定哪些事不能对用户许诺、决定第几版色块方案要推翻重来、决定哪些功能宁可不做。** 这些判断——需求怎么定义、边界怎么守、界面为什么长成这样——是我做了多年产品经理之后，唯一真正带进这个项目的东西，也是 AI 目前不会替我负责的部分。

图麦笔记的创新不在模型，而在**重新设计了知识记录的起点**：不再等待用户写下第一句话，而是从他已经发现的价值开始。收藏工具让你存下更多，笔记工具让你写得更认真，我们想做的是中间那一层——**让每一次随手保存，都有机会成为未来真正用得上的积累。**

> **图麦笔记，让收藏不止于收藏，让信息逐渐成为自己的知识。**